BITS Pilani - Work Integrated Learning Programmes | Machine Learning Operations (MLOps) - AIMLCZG523

Heart Disease Risk Prediction

End-to-end ML model development, CI/CD, containerised deployment and monitoring - Assignment 01 Project Report (50 marks)

Student name	Divvela Karthik	BITS ID / Roll no.	2025AE05849
Course	MLOPS-- AIMLCZG523	Submission date	20/OCT/2026
Code repository	https://github.com/karthikdivvela/Heartdisease	Demo video	https://drive.google.com/file/d/12ClKhZ_BUU53sPFQW_bvJB-7zr2UEPMo/view?usp=sharing
Deployed API / access	Not public. Local: http://localhost:8000/docs (Docker); lab LoadBalancer: http://10.103.55.10/docs (Section 9.4)		

1. Introduction and objectives

The goal is to design, build and deploy a reproducible machine learning solution using MLOps practices: automated data preparation, experiment tracking, tested and linted code, a CI pipeline, a Docker image, a Kubernetes deployment and basic monitoring. The business problem is to predict the risk of heart disease from patient health data and serve the prediction as a monitored REST API (a learning project, not a medical device).

The solution uses the Heart Disease UCI (Cleveland) dataset. Two classifiers (Logistic Regression and Random Forest) were tuned with cross-validation, tracked in MLflow, packaged as one scikit-learn pipeline, served with FastAPI in a Docker container, deployed to a local Minikube cluster behind a LoadBalancer service, and monitored with Prometheus and Grafana.

1.1 Repository layout

Path	Purpose
scripts/, src/	download_data.py and create_grafana_dashboard.py; data.py (cleaning), features.py (preprocessing), eda.py (plots), train.py (training, tuning, MLflow)
app/, tests/	main.py - FastAPI service (/predict, /health, /metrics, request logging); pytest tests for data, training and API (8 tests)
data/, models/, reports/	cleaned heart.csv; model.joblib and metrics.json; EDA and evaluation plots
Dockerfile, requirements*.txt	container build; pinned dependencies (full and slim serving sets)
k8s/, monitoring/, .github/workflows/ci.yml	Kubernetes manifests; docker-compose.yml and prometheus.yml for Prometheus/Grafana; GitHub Actions pipeline
screenshots/	evidence used in this report

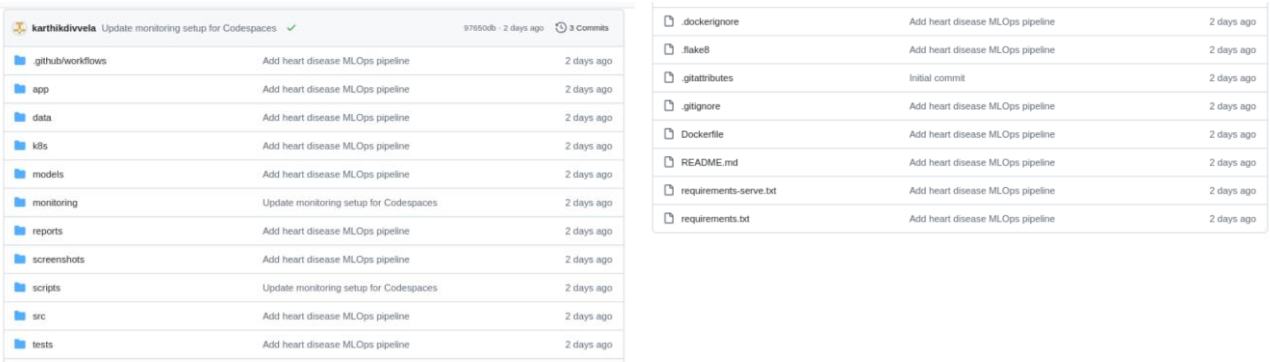


Figure 1: GitHub repository home page.

2. Setup and installation instructions

Prerequisites: Python (3.12 on the lab machine; CI and the Docker image use 3.14), Git and Docker; Minikube for Kubernetes, with kubectl run through Minikube (alias kubectl='minikube kubectl --'). Everything (training, API, Docker, Minikube, monitoring) was run on the college virtual lab machine (Linux); the same commands work on any machine with Docker.

```
git clone https://github.com/karthikdivvela/Heartdisease.git
cd Heartdisease
python -m venv .venv
source .venv/bin/activate   # Windows:
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/download_data.py   # data/heart_raw.csv
python -m src.data     # data/heart.csv (cleaned)
python -m src.eda      # reports/eda/*.png
python -m src.train    # model.joblib + MLflow runs
mlflow ui --backend-store-uri sqlite:///mlflow.db
pytest -v ; flake8 src app tests scripts
uvicorn app.main:app --reload   # :8000/docs
```

```
docker build -t heart-api:latest .
docker run -d -p 8000:8000 --name heart-api heart-api:latest

minikube start --driver=docker --cni=bridge
minikube image load heart-api:latest
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
kubectl get pods,svc ; minikube tunnel   # 2nd terminal

# monitoring: API + Prometheus + Grafana (host network)
docker rm -f heart-api   # free port 8000 first
docker compose -f monitoring/docker-compose.yml up -d
```

3. Task 1 - Data acquisition and exploratory data analysis

The Cleveland subset has 303 patient records with 13 clinical features. The original target (0-4) was converted to a binary label (0 = no disease, 1 = disease). Numeric features: age, trestbps, chol, thalach, oldpeak. Categorical features: sex, cp, fbs, restecg, exang, slope, ca, thal.

- scripts/download_data.py downloads processed.cleveland.data from UCI (ucimlrepo as fallback) and saves data/heart_raw.csv.
- src/data.py turns the ? placeholders into missing values (ca: 4 rows, thal: 2 rows), converts columns to numeric, binarises the target, drops duplicates and writes data/heart.csv.
- Missing values are not filled here but inside the scikit-learn pipeline (median for numeric, most frequent for categorical), so imputation statistics come from training folds only and cannot leak test information.

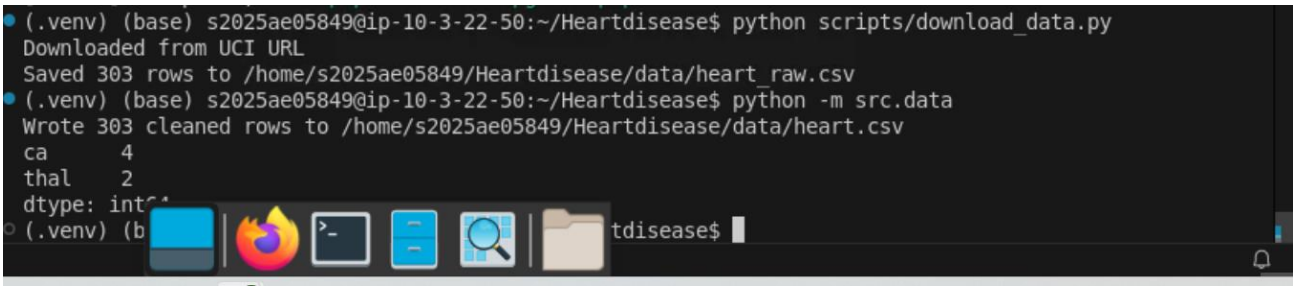


Figure 2: Data download and cleaning output.

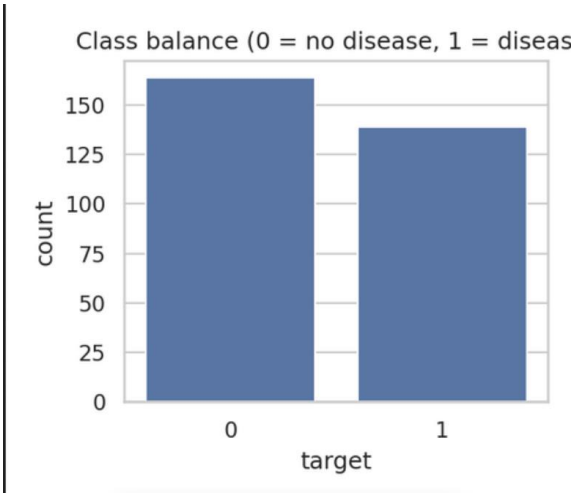


Figure 3: Class balance of the target.

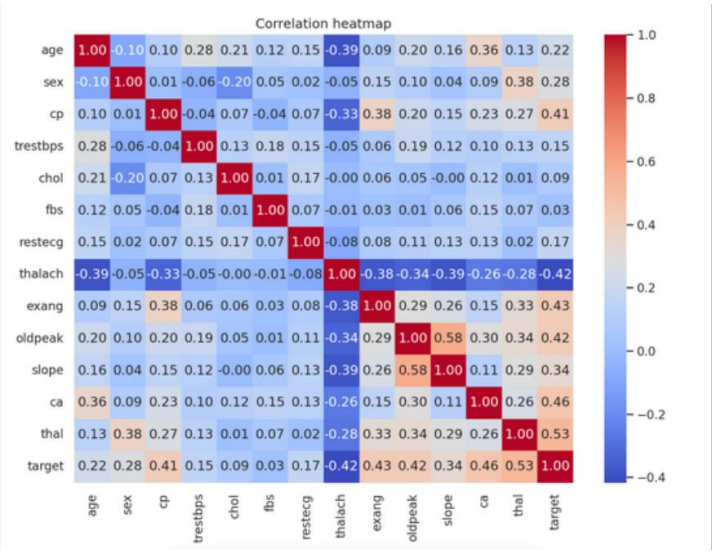


Figure 5: Correlation heatmap.

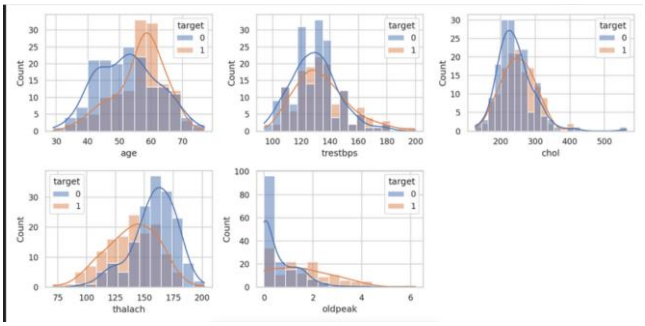


Figure 4: Histograms of numeric features by target.

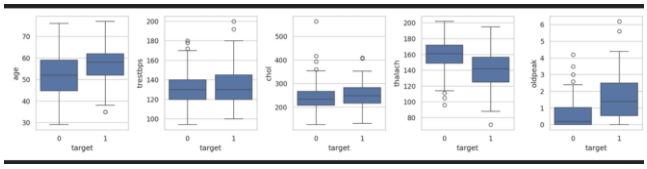


Figure 6: Boxplots of numeric features by target.

Class balance. About 164 patients without heart disease and 139 with it (roughly 54% / 46%): close to balanced, so a model cannot score well by predicting the majority class, and accuracy, precision, recall and ROC-AUC are all meaningful.

Distributions. Maximum heart rate (thalach) is clearly lower and ST depression (oldpeak) higher in patients with disease; age is shifted slightly upward. trestbps and chol overlap heavily between classes. oldpeak is right-skewed with many zeros and chol has a long right tail.

Correlations. Most correlated with the target: thal (0.53), ca (0.46), exang (0.43), oldpeak (0.42), cp (0.41), and thalach (-0.42). The strongest feature pair is slope-oldpeak (0.58), then age-thalach (-0.39); no pair is strong enough to be a serious multicollinearity problem.

Outliers. One cholesterol value near 560, resting blood pressure up to about 200 and oldpeak up to about 6. They are clinically possible and the dataset is small, so no rows were removed; scaling, L2 regularisation and the tree-based forest limit their influence.

4. Task 2 - Feature engineering and model development

Features. src/features.py builds one ColumnTransformer used for both training and serving: numeric features get median imputation and standard scaling; categorical features get most-frequent imputation and one-hot encoding with handle_unknown="ignore", so unseen categories do not crash the API. No additional engineered features were added; the 13 original features are used as they are.

Protocol. 80/20 stratified split (seed 42: 242 training and 61 test rows). Each model was tuned with GridSearchCV (stratified 5-fold, scoring ROC-AUC), then evaluated with 5-fold cross-validation (accuracy, precision, recall, F1, ROC-AUC) and finally on the untouched test set. Logistic Regression grid: C in {0.01, 0.1, 1, 10}, L2 penalty. Random Forest grid: n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 3}.

Metric	Logistic Regression	Random Forest
CV accuracy / precision / recall	0.847 / 0.877 / 0.783	0.806 / 0.811 / 0.764
CV ROC-AUC (mean)	0.902	0.901
Test ROC-AUC (accuracy / recall)	0.966 (0.885 / 0.929)	0.947 (0.869 / 0.893)
Best parameters	C = 1, penalty = l2	max_depth = 5, min_samples_leaf = 3, n_estimators = 100

Model selection. Logistic regression was selected because it had the highest mean cross-validated ROC-AUC (0.902). Random forest was almost identical (0.901), a difference far smaller than the fold-to-fold spread (0.014 and 0.024), so the models are effectively tied on that metric; logistic regression was ahead on CV accuracy, precision and recall and on the test set (ROC-AUC 0.966 vs 0.947, recall 0.929 vs 0.893). Test scores exceed CV scores, but the test set has only 61 patients, so that gap is within normal noise. For a clinical risk tool a simpler, interpretable model is attractive because its coefficients can be inspected and explained. Its confusion matrix shows 28 true negatives, 5 false positives, 2 false negatives and 26 true positives.

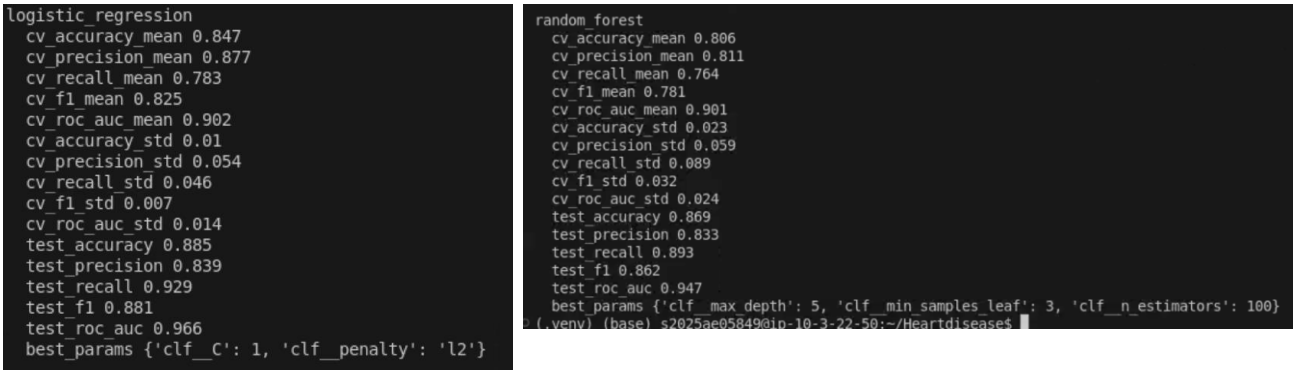


Figure 7: Training output (CV and test metrics) for logistic regression and random forest.

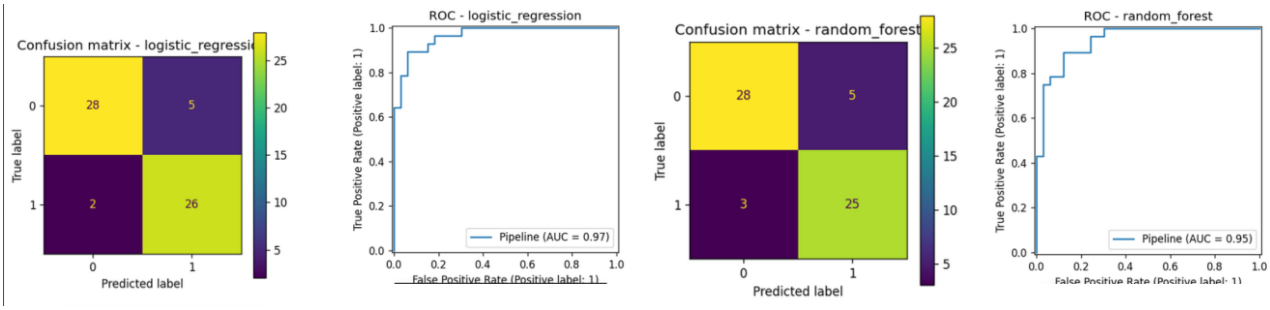


Figure 8: Confusion matrix and ROC curve of logistic regression (left pair) and random forest (right pair).

5. Task 3 - Experiment tracking with MLflow

src/train.py logs each run to a local MLflow store (sqlite:///mlflow.db, experiment heart-disease), one run per model. Logged: model name, training-set size and best hyper-parameters; cross-validation metrics (mean and standard deviation) and test metrics; the ROC and confusion-matrix plots (plots/); and the full pipeline with an input/output signature. The comparison view shows logistic regression ahead of random forest on every cross-validation and test metric with a smaller spread across folds, which confirmed the choice and lets the final model be traced back to the run that produced it.



Figure 9: MLflow experiment page with both runs.



Figure 10: Parameters and metrics of the logistic regression run.

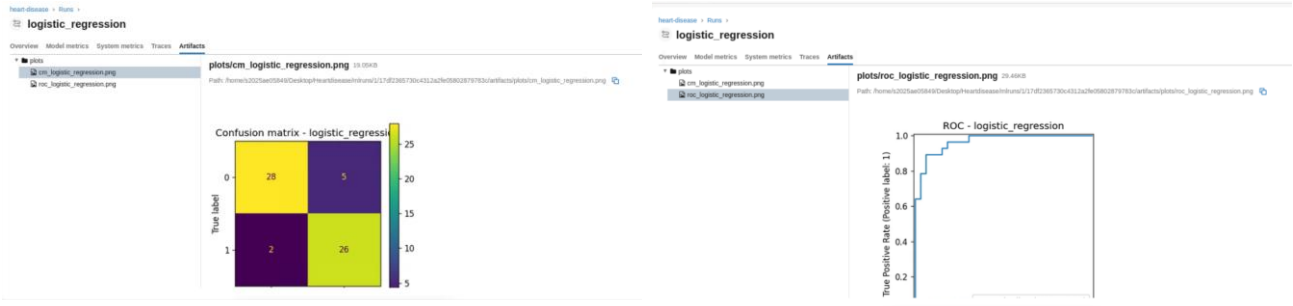


Figure 11: Artifacts logged for a run (confusion matrix and ROC plots).

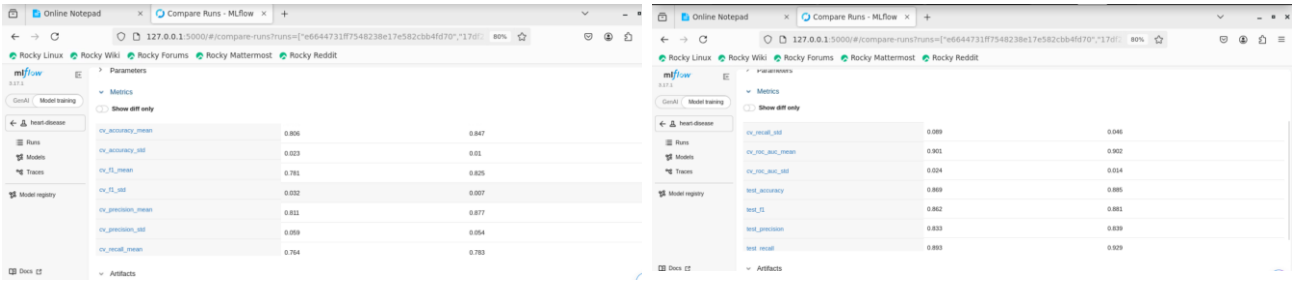


Figure 12: Comparison of the two runs (CV and test metrics).

6. Task 4 - Model packaging and reproducibility

- **Format.** The selected model is one scikit-learn Pipeline (preprocessing + classifier) in models/model.joblib, so the API applies exactly the training transformations. The same pipeline is logged to MLflow with a signature; models/metrics.json stores the selected model, features, metrics and best parameters.
- **Dependencies and determinism.** requirements.txt pins every development package; requirements-serve.txt is the slim set used in the Docker image. Random seed 42 is used for the split, CV folds and models.
- **Inference script.** src/predict.py loads models/model.joblib and predicts for a JSON or CSV file (python -m src.predict --input samples/sample_patients.csv), printing prediction, label, confidence and probability_disease exactly as the API does. Sample inputs are in samples/.
- **Clean-setup check.** CI installs requirements.txt on a fresh runner, runs tests and retrains the model from data/heart.csv.

7. Task 5 - CI/CD pipeline and automated testing

Test file	What is verified
tests/test_data.py (3)	cleaning handles ? values and binarises the target; cleaning is idempotent; the preprocessor outputs a finite matrix
tests/test_model.py (1)	end-to-end training on a small dataset produces model.joblib and metrics.json and valid probabilities
tests/test_api.py (4)	/health; /predict returns prediction and confidence; invalid or missing input returns HTTP 422; /metrics exposes counters

.github/workflows/ci.yml runs on every push to main and on pull requests. Job build-test-train: checkout, Python 3.14 with pip cache, install dependencies, flake8 lint, pytest (JUnit XML), model training, then upload artifacts (model, metrics, plots, JUnit report, MLflow database) even if an earlier step failed. Job docker (after the first succeeds): build the image, start the container, wait for /health, send a sample /predict request and print the logs.

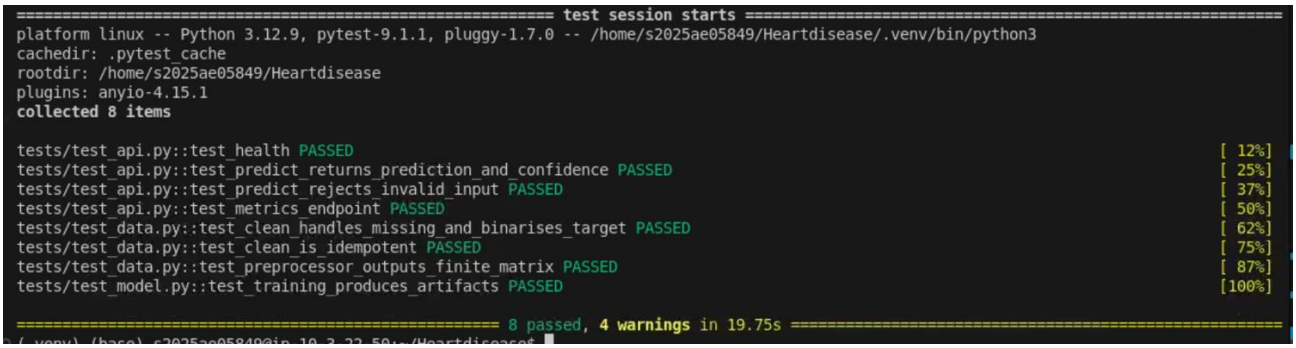


Figure 13: Local pytest result (8 passed).

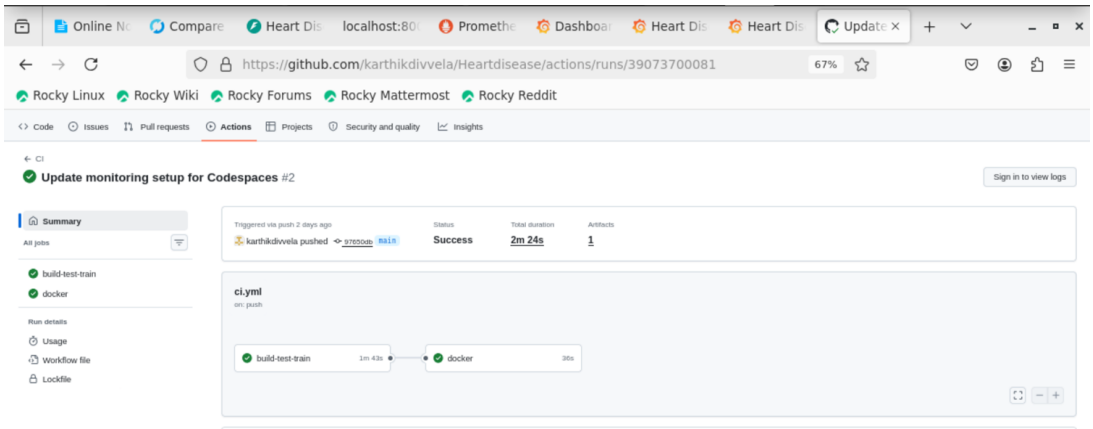


Figure 14: Successful workflow run (both jobs green).

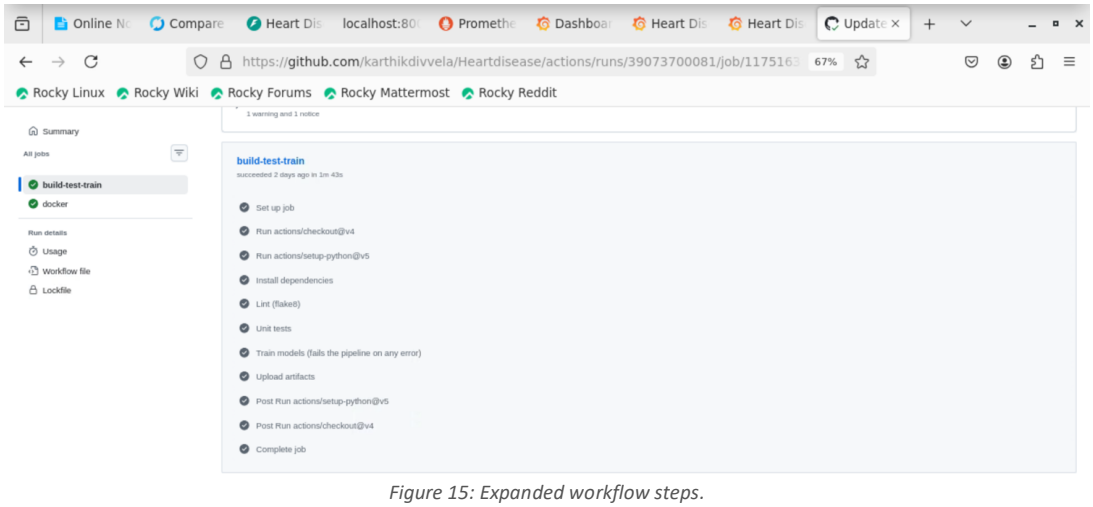


Figure 15: Expanded workflow steps.

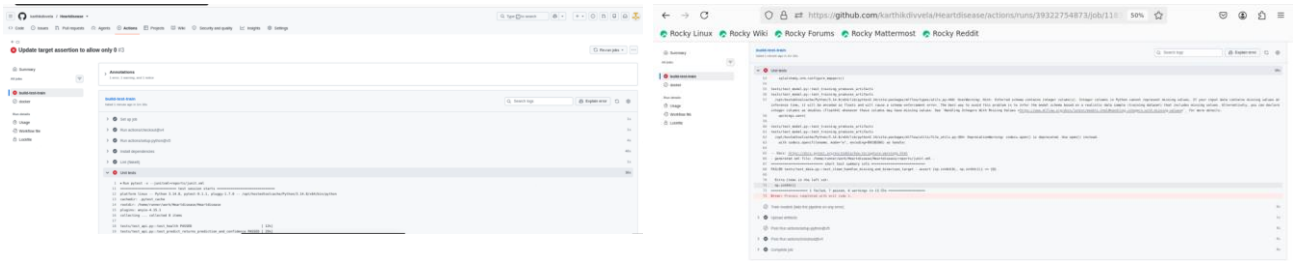


Figure 16: Pipeline failing on a broken test (Unit tests step: 1 failed, 7 passed).

Failure demonstration. I deliberately changed the assertion in test_clean_handles_missing_and_binarises_target so that it only allowed the value 0. The pipeline failed at the Unit tests step (1 failed, 7 passed), the training and Docker jobs did not run, and after the assertion was restored the run turned green again.

8. Task 6 - Model containerisation

The image is built from python:3.14-slim. It installs only requirements-serve.txt, copies app/, src/ (needed to unpickle the pipeline) and models/model.joblib, runs as a non-root user, defines a HEALTHCHECK on /health and starts uvicorn on port 8000.

Endpoint	Method	Description
/predict	POST	JSON with the 13 features; returns prediction (0/1), label, confidence and probability_disease. Inputs are validated (ranges, required fields).
/health, /metrics, /docs	GET	liveness/readiness (model file exists); Prometheus metrics; interactive OpenAPI documentation

```
curl -X POST localhost:8000/predict -H "Content-Type: application/json" -d
'{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0
,"thal":6}'
-> {"prediction":0,"label":"no disease","confidence":0.8345,"probability_disease":0.1655}
```

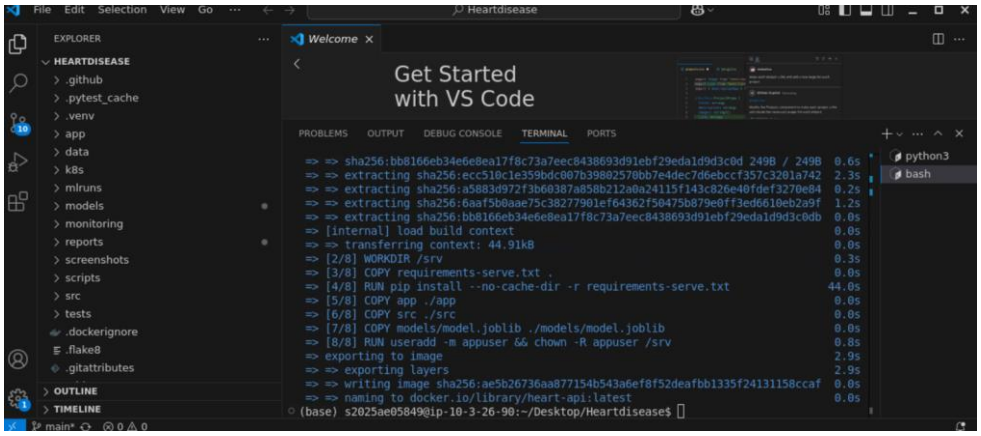


Figure 17: Docker image build.

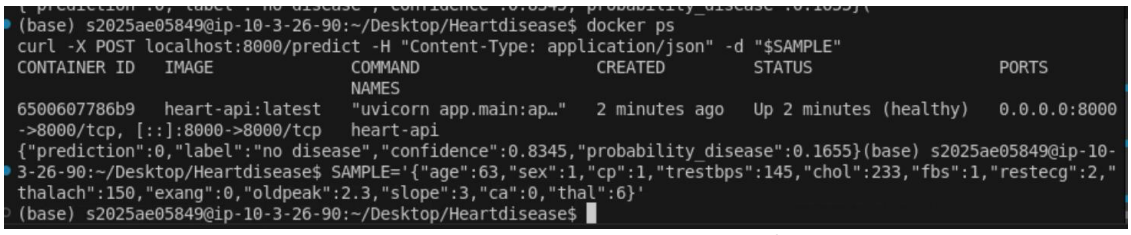


Figure 18: Container running (healthy) and answering a sample /predict request.

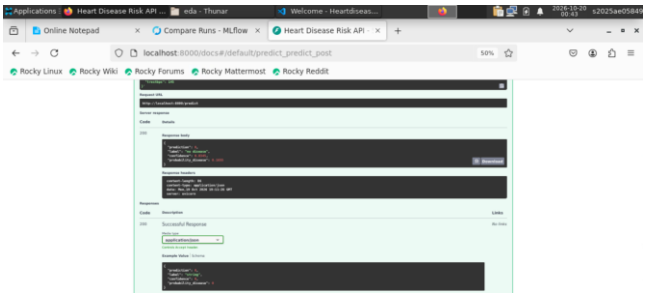


Figure 19: Interactive API documentation (/predict response).



Figure 20: Input validation (age = -5 returns HTTP 422).

9. Task 7 - Production deployment on Kubernetes

Manifests. k8s/deployment.yaml: 2 replicas of heart-api:latest, readiness and liveness probes on /health, resource requests (100m CPU, 128Mi) and limits (500m CPU, 512Mi), Prometheus scrape annotations. k8s/service.yaml: Service of type LoadBalancer on port 80 forwarding to container port 8000. k8s/ingress.yaml: optional Ingress alternative.

Deployment and verification. A local cluster was created with Minikube (Docker driver) on the college lab machine. The image was loaded with minikube image load and the manifests applied; both pods reached 1/1 Running on a Ready node. The service showed EXTERNAL-IP <pending> until minikube tunnel was started in a second terminal, after which it received 10.103.55.10. Through that address /health returned status ok with model_loaded true, and /predict returned prediction 0 (no disease) with confidence 0.8345.

Access. The API is not public. In the college lab it is reachable through the LoadBalancer at http://10.103.55.10/docs (while minikube tunnel runs) and directly from the container at http://localhost:8000/docs. To test elsewhere, follow Section 2 and open either address.

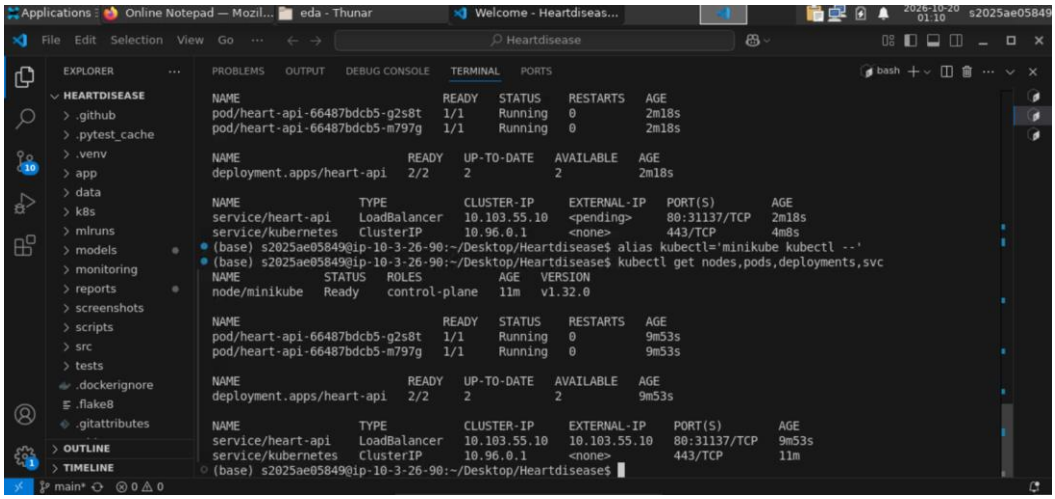


Figure 21: Deployed pods and service (EXTERNAL-IP 10.103.55.10 assigned).

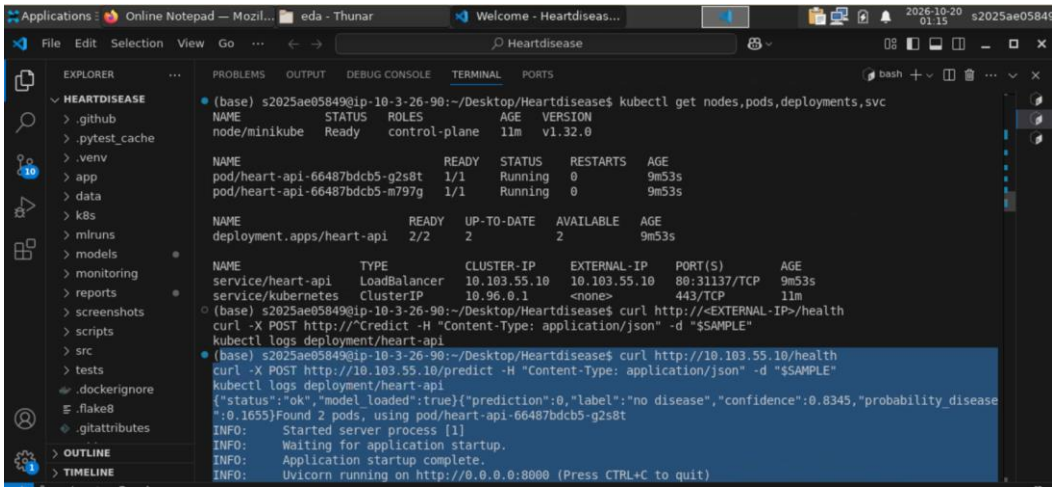


Figure 22: Endpoints verified through the LoadBalancer.

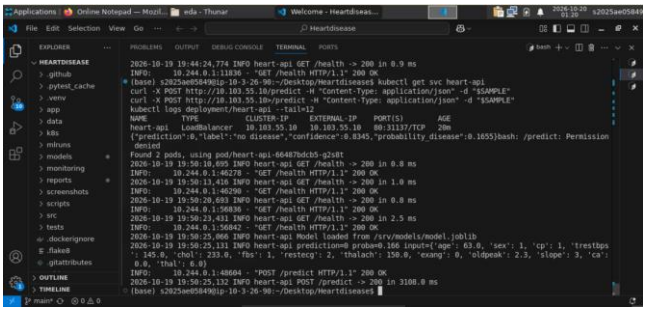


Figure 23: Pod logs.

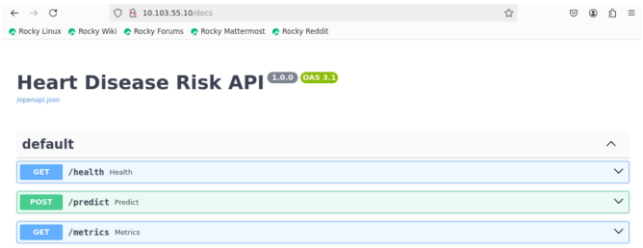


Figure 24: API reachable via the LoadBalancer address.

10. Task 8 - Monitoring and logging

Logging. A FastAPI middleware logs every request (method, path, status, latency in ms) and each prediction with its probability and input, to stdout, so docker logs and kubectl logs show them.

Metric	Type	Use
api_requests_total{endpoint,status}	counter	request rate and error rate
api_request_latency_seconds{endpoint}	histogram	latency percentiles (p95)
predictions_total{label}	counter	predictions per class (basic drift signal)

Prometheus scrapes /metrics every 5 seconds. Grafana shows four panels created by scripts/create_grafana_dashboard.py: requests per second, predictions by class, p95 latency of /predict and total requests. monitoring/docker-compose.yml starts the API, Prometheus and Grafana with host networking (tested on Linux), so Prometheus scrapes the API at localhost:8000; Docker Desktop on Windows/macOS does not support host networking.

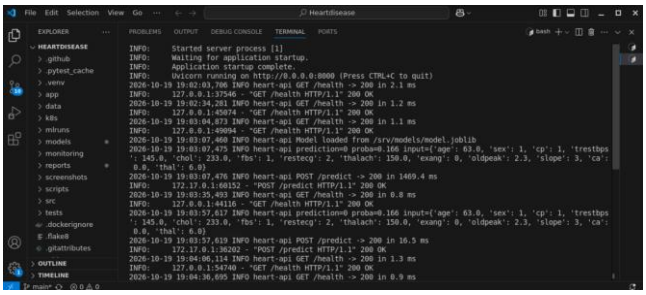


Figure 25: API request logs.

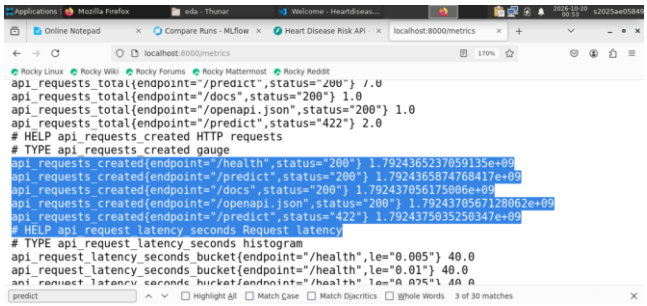


Figure 26: Metrics endpoint.

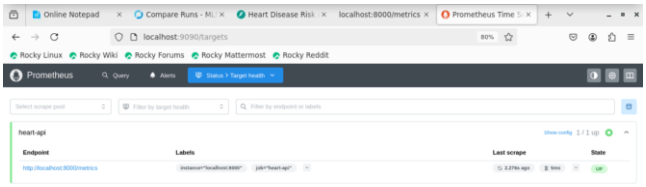


Figure 27: Prometheus target is UP.

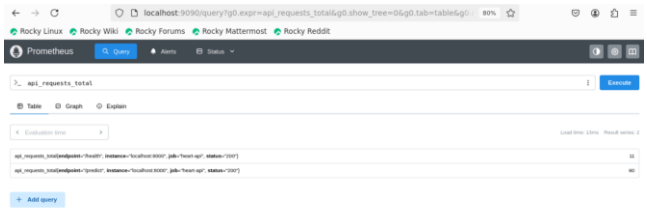
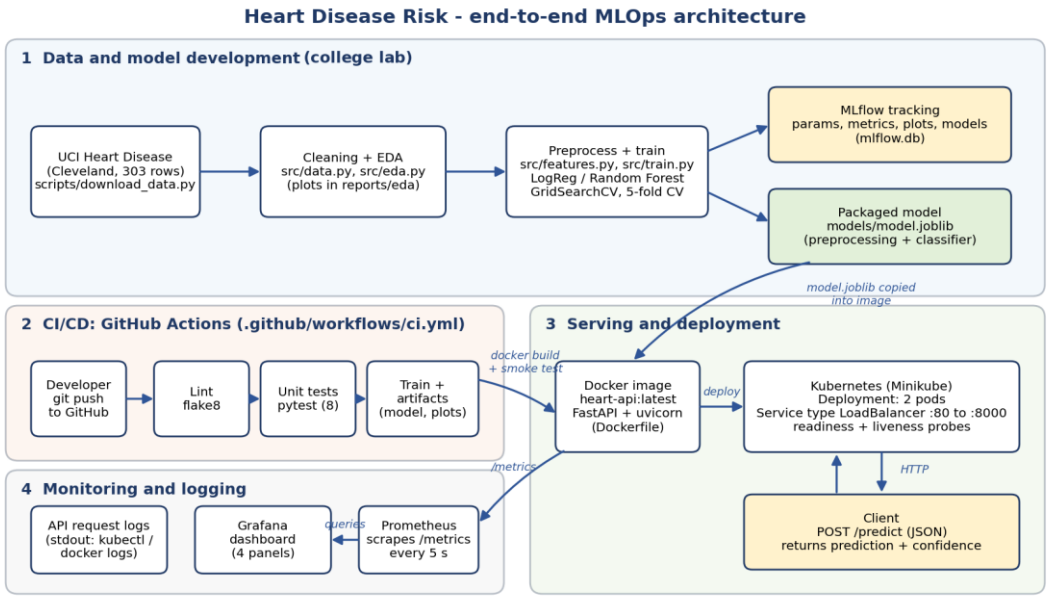


Figure 28: Prometheus query.



Figure 29: Grafana dashboard (requests/s, predictions by class, p95 latency, total requests).

11. Architecture



Architecture: data and model development, CI/CD, serving and deployment, monitoring.

12. Production-readiness requirements

Requirement from the brief	How it is met

Scripts run from a clean setup using the requirements file	Pinned requirements.txt; CI installs it on a fresh runner and runs tests and training (CI summary, Section 2)
Model serves correctly in an isolated environment (Docker proof)	Image builds and answers /health and /predict; also smoke-tested in the CI docker job (Figures 17-18)
Pipeline fails on code or test errors with clear logs	Lint and tests have no continue-on-error; a broken test turned the run red (Figure 16)

13. Challenges and how they were solved

Problem	Cause	Resolution
EXTERNAL-IP stayed <pending>	Minikube assigns no external IP to a LoadBalancer unless a tunnel runs	started minikube tunnel in a second terminal; the service then got 10.103.55.10
kubectl used through Minikube	kubectl was not a separate command on the lab machine	aliased kubectl to 'minikube kubectl --'; manifests and get/logs commands work unchanged
Python version differs between lab and CI	lab runs Python 3.12, CI runner and Docker image use 3.14	same pinned library versions; model confirmed to load in CI and the container (/health: model_loaded true)
Prometheus could not use localhost to reach the API	Prometheus runs in a container, so localhost points to itself	compose file runs Prometheus on the host network so it scrapes localhost:8000 (Linux only)

14. Conclusion and future work

This project turned the UCI Cleveland data into a complete MLOps pipeline: cleaning and EDA, tuned logistic regression and random forest models tracked in MLflow, a single packaged pipeline, a FastAPI service in Docker, a two-pod Kubernetes deployment and Prometheus/Grafana monitoring. The selected logistic regression model reached a test ROC-AUC of 0.966 (accuracy 0.885, recall 0.929). Automating lint, tests, training and the Docker smoke test in GitHub Actions showed how much a pinned environment and a pipeline that fails loudly help reproducibility. **Limitations:** about 300 patients (the test set has 61), a fixed 0.5 threshold, and a learning exercise rather than a medical device. **Future work:** a managed cloud cluster (GKE/EKS/AKS) with a Helm chart, pushing the image to a registry from CI, an MLflow model registry, and drift monitoring with alerts.

Deliverables: code, Dockerfile and requirements (repository root); data/heart.csv and scripts/download_data.py; src/eda.py, src/train.py and src/predict.py (inference); samples/; tests/; .github/workflows/ci.yml; k8s/; screenshots/; demo video (link on the title page). **References:** UCI Machine Learning Repository, Heart Disease dataset (Detrano et al., 1989); scikit-learn, MLflow, FastAPI, Prometheus, Grafana, Docker and Kubernetes/Minikube documentation.